📜 LearnJS Master Curriculum Taxonomy

Production Reference Architecture | *Comprehensive Guide from Zero to Engine Internals*

📊 Overview & Curriculum Metadata

Metric	Details
Total Modules	10 Master Levels
Track Focus	Native JavaScript, Runtime Mechanics, System Architecture & Tooling
Core Modules	CORE — Essential production capability
Advanced Extensions	OPTIONAL — Specialized domain knowledge

📑 Table of Contents

🚀 Level 1: Beginner Foundations & Practical Syntax

1.1 First Steps: What is JavaScript & Writing Your First Code CORE

- **1.1.1** What JavaScript Does on the Web
- **1.1.2** Running Code in Browser Developer Console (console.log)
- **1.1.3** Embedding Scripts in HTML (<script> Tags)
- **1.1.4** Sequential Code Execution Basics

1.2 Environment Setup (VS Code, Terminal & Node Runner) CORE

- **1.2.1** VS Code Installation & Workspace Setup
- **1.2.2** Using Integrated Terminals (Bash / Zsh / WSL)
- **1.2.3** Running Scripts Locally via Node.js (node script.js)
- **1.2.4** Basic Git Version Control Initialization (git init, git commit)

1.3 Storing Data: Variables (let, const, var Intro) CORE

- **1.3.1** Declaring Variables with let and const
- **1.3.2** Variable Reassignment & Identifier Naming (camelCase)
- **1.3.3** Why var Should Be Avoided in Modern Code
- **1.3.4** Basic Mutability Concepts with const

1.4 Primitive Data Types & Numeric Formatting CORE

- **1.4.1** Working with
  Primitives: string, number, boolean, null, undefined, symbol, bigint
- **1.4.2** Checking Types using the typeof Operator
- **1.4.3** Using Numeric Separators for Readability (1_000_000)
- **1.4.4** Modern Template Literals (`Hello ${name}`)

1.5 Operators, Short-Circuiting & Modern Expressions CORE

- **1.5.1** Arithmetic (+, -, *, /, %) & Assignment Operators (+=, -=)
- **1.5.2** Comparison Operators (<, >, <=, >=) & Strict Equality (=== vs ==)
- **1.5.3** Logical Operators (&&, ||, !) & Short-Circuit Evaluation
- **1.5.4** Modern Expression Operators: Nullish Coalescing (??) & Optional Chaining (?.)
- **1.5.5** Logical Assignment Operators (&&=, ||=, ??=)
- **1.5.6** Explicit Type Conversion (Number(), String(), Boolean())

1.6 Control Flow & Foundational Error Handling CORE

- **1.6.1** Branching Logic with if, else if, and else
- **1.6.2** Truthy and Falsy Value Evaluation
- **1.6.3** Compact Conditionals with the Ternary Operator (? :)
- **1.6.4** Multi-branch Conditionals with switch Statements
- **1.6.5** Catching Runtime Exceptions with try...catch...finally Blocks
- **1.6.6** Throwing Explicit Exceptions using throw new Error()

🧱 Level 2: Core Data Structures & Functional Building Blocks

2.1 Reusable Code: Functions & Parameters CORE

- **2.1.1** Defining & Calling Function Declarations
- **2.1.2** Function Expressions vs Declarations
- **2.1.3** Passing Arguments & Setting Default Parameters
- **2.1.4** Rest Parameters (...args) for Variable Arguments
- **2.1.5** Returning Values with return (vs Implicit undefined)

2.2 Modern Function Syntax: Arrow Functions CORE

- **2.2.1** Concise Arrow Function Syntax (() => {})
- **2.2.2** Implicit Returns for Single-Line Expressions
- **2.2.3** Arrow Functions as Callbacks

2.3 Repetition & Loops (for, while, for...of, for...in) CORE

- **2.3.1** Imperative Iteration with for Loops
- **2.3.2** Condition-Based Iteration with while & do...while Loops
- **2.3.3** Controlling Loop Flow: break and continue
- **2.3.4** Iterating Array Values with for...of Loops
- **2.3.5** Enumerating Object Keys with for...in Loops

2.4 Lists of Data: Arrays & Index Operations CORE

- **2.4.1** Creating Arrays & Zero-Indexed Access
- **2.4.2** Adding/Removing Elements (push, pop, shift, unshift)

- **2.4.3** Splicing & Slicing Arrays (splice, slice)
- **2.4.4** Array Destructuring & Spread Operator Syntax (...)

2.5 Processing Data: Higher-Order Array Methods CORE

- **2.5.1** Iterating Elements with forEach
- **2.5.2** Transforming Datasets with map
- **2.5.3** Filtering Data with filter
- **2.5.4** Accumulating Values with reduce
- **2.5.5** Searching Arrays: find, findIndex, includes, some, every
- **2.5.6** Sorting Arrays with Custom Comparators (sort, toSorted)

2.6 Structured Data: Objects, JSON & The URL API CORE

- **2.6.1** Object Literals & Key-Value Pair Operations
- **2.6.2** Dot Notation vs Dynamic Bracket Notation Access
- **2.6.3** Object Destructuring & Property Shorthands
- **2.6.4** Object Utilities: Object.keys(), Object.values(), Object.entries()
- **2.6.5** Parsing & Stringifying JSON Data (JSON.parse, JSON.stringify)
- **2.6.6** Working with Web URLs via URL & URLSearchParams APIs

---

🌐 **Level 3: Interactive Web Development & Essential Browser APIs**

3.1 The DOM Tree & Selecting Elements CORE

- **3.1.1** Understanding Document Object Model (DOM) Architecture
- **3.1.2** Selecting Elements with querySelector & querySelectorAll
- **3.1.3** Legacy Selectors: getElementById, getElementsByClassName
- **3.1.4** Static NodeLists vs Live HTMLCollections

3.2 Dynamic Web Pages: Modifying Content & Attributes CORE

- **3.2.1** Creating & Appending Elements (createElement, append, appendChild)
- **3.2.2** Modifying Text Safely with textContent (vs innerHTML Risks)
- **3.2.3** Dynamic Styling: Class Manipulation via classList (add, remove, toggle)
- **3.2.4** Reading & Writing HTML Attributes (getAttribute, setAttribute, dataset)

- **3.2.5** Removing DOM Nodes (remove, removeChild)

3.3 User Interactions: Browser Event Mechanics CORE

- **3.3.1** Attaching Event Listeners (addEventListener)
- **3.3.2** Handling Mouse, Keyboard, and Form Input Events
- **3.3.3** Working with the Event Object (event.target, event.key)
- **3.3.4** Preventing Default Browser Behaviors (event.preventDefault())

3.4 Event Propagation: Bubbling, Capturing & Delegation CORE

- **3.4.1** Understanding Event Propagation: Capturing vs Bubbling
- **3.4.2** Stopping Event Propagation (stopPropagation)
- **3.4.3** Implementing the Event Delegation Pattern for Dynamic Lists

3.5 Web Utility APIs: Clipboard, Geolocation & Notifications OPTIONAL

- **3.5.1** Reading/Writing System Clipboard Data (navigator.clipboard)
- **3.5.2** Retrieving Device Coordinates via Geolocation API
- **3.5.3** Displaying System Notifications via Notification API

3.6 Browser Storage Engines: localStorage & sessionStorage CORE

- **3.6.1** Persistent Client-Side Storage with localStorage
- **3.6.2** Session-Scoped Storage with sessionStorage
- **3.6.3** Persisting Complex Objects using JSON Serialization

---

⚙️ **Level 4: Under-the-Hood Runtime Mechanics**

4.1 How Code Executes: Call Stack & Execution Context CORE

- **4.1.1** Creation Phase vs Execution Phase in JavaScript Engines
- **4.1.2** Global Execution Context vs Function Execution Context
- **4.1.3** Stack Frames & Call Stack Execution Tracing
- **4.1.4** Stack Overflow Errors & Recursion Limits

4.2 Scope Chains, Hoisting & The Temporal Dead Zone (TDZ) CORE

- **4.2.1** Scope Levels: Global, Function, and Block Scope
- **4.2.2** Lexical Scope Lookup Chains

- **4.2.3** Function & Variable Hoisting Mechanics (var vs let/const)
- **4.2.4** The Temporal Dead Zone (TDZ) & Reference Errors

4.3 Closures & Encapsulated State CORE

- **4.3.1** Retaining Lexical Scope via Function Closures
- **4.3.2** Data Encapsulation & Simulating Private State
- **4.3.3** Practical Closure Use Cases: Factory Functions & Memoization

4.4 Dynamic Context: The this Keyword CORE

- **4.4.1** Implicit Binding Rules in Object Methods
- **4.4.2** Explicit Binding: call(), apply(), and bind()
- **4.4.3** Constructor Function Binding with the new Operator
- **4.4.4** Lexical this Binding in Arrow Functions

4.5 Prototype Chains & Inheritance Mechanics CORE

- **4.5.1** Prototype Objects & [[Prototype]] Internal Links
- **4.5.2** Prototype Chain Property Lookups & Shadowing
- **4.5.3** Function .prototype vs Instance __proto__
- **4.5.4** Pure Prototypal Inheritance using Object.create()

4.6 Object-Oriented Programming: ES6 Class Syntax CORE

- **4.6.1** Class Declarations, Constructors & Instantiation
- **4.6.2** Instance Methods, Getters, Setters & Static Methods
- **4.6.3** Private Class Fields (#private) & Static Blocks
- **4.6.4** Inheritance Patterns: extends, super, and Method Overriding

4.7 Functional Programming (FP) Core Principles CORE

- **4.7.1** Pure Functions & Eliminating Side Effects
- **4.7.2** Immutability Patterns in Application State
- **4.7.3** Higher-Order Functions & Currying Patterns
- **4.7.4** Function Composition & Pipeline Concepts

⚡ **Level 5: Asynchronous JavaScript & Network Communications**

5.1 Synchronous vs Asynchronous Execution Concepts CORE

- **5.1.1** Single-Threaded Constraints & Non-Blocking I/O
- **5.1.2** CPU-Bound vs I/O-Bound Operations
- **5.1.3** Basic Async Timers: setTimeout & setInterval

5.2 Asynchronous Callbacks & Callback Hell CORE

- **5.2.1** Asynchronous Callback Execution Patterns
- **5.2.2** Node.js Error-First Callback Conventions
- **5.2.3** Nesting Issues, Inverted Control & Callback Hell

5.3 Promises Engine & State Lifecycle CORE

- **5.3.1** Promise States: Pending, Fulfilled, Rejected
- **5.3.2** Creating Promises with the new Promise Constructor
- **5.3.3** Promise Chaining: .then(), .catch(), and .finally()
- **5.3.4** Promise
  Utilities: Promise.resolve(), Promise.reject(), Promise.withResolvers()

5.4 Modern Async Syntax: async & await CORE

- **5.4.1** async Function Declarations & Implicit Promises
- **5.4.2** Pausing Execution with the await Operator
- **5.4.3** Structured Error Handling with try…catch…finally Blocks
- **5.4.4** Top-Level await in Modern ES Modules

5.5 Working with Web APIs: Fetch API & REST CORE

- **5.5.1** Client-Server Communication & RESTful API Architecture
- **5.5.2** Sending Requests with fetch() & Parsing Responses (.json())
- **5.5.3** Configuring HTTP Verbs: GET, POST, PUT, DELETE
- **5.5.4** Handling Request Headers & Payload Data
- **5.5.5** Request Cancellation via AbortController API

5.6 Managing Parallel Async Requests (Promise Concurrency) CORE

- **5.6.1** Parallel Execution with Promise.all()
- **5.6.2** Fault-Tolerant Concurrent Operations with Promise.allSettled()

- **5.6.3** Handling Race Conditions with Promise.race() & Promise.any()

🛠️ Level 6: Developer Tooling, Ecosystem & Testing

6.1 JavaScript Module Systems (CJS vs ES Modules) CORE

- **6.1.1** Code Encapsulation History: Script Tags & IIFEs
- **6.1.2** CommonJS Modules (require, module.exports)
- **6.1.3** Native ES Modules (import, export, Named/Default)
- **6.1.4** Dynamic Imports using import() Expressions

6.2 Package Management & Monorepo Basics (npm, pnpm) CORE

- **6.2.1** Dependency Management via package.json & Lockfiles
- **6.2.2** Semantic Versioning Rules (SemVer)
- **6.2.3** Package Execution: npm, pnpm, bun Performance Differences
- **6.2.4** Monorepo Workspace Configurations

6.3 Automated Unit Testing & TDD (Vitest / Jest) CORE

- **6.3.1** Test-Driven Development (TDD) Core Cycles
- **6.3.2** Writing Test Suites: describe, it, expect Assertions
- **6.3.3** Testing Asynchronous Logic & Promise Rejections
- **6.3.4** DOM Testing using JSDOM / HappyDOM Environments

6.4 Professional Visual Debugging Workflows CORE

- **6.4.1** Advanced DevTools: Conditional Breakpoints & Logpoints
- **6.4.2** DOM Mutation & Event Listener Breakpoints
- **6.4.3** Inspecting Call Stacks & Watching Scope Variables

6.5 Code Quality Automation: ESLint & Prettier CORE

- **6.5.1** Static Linting Rule Configurations with ESLint
- **6.5.2** Automated Code Formatting with Prettier
- **6.5.3** Pre-commit Hook Automation via Husky & lint-staged

---

🔮 Level 7: Advanced Language Capabilities & Metaprogramming

7.1 Advanced Error Handling & Custom Error Classes CORE

- **7.1.1** Subclassing the Native Error Object
- **7.1.2** Building Domain Errors (ValidationError, NetworkError)
- **7.1.3** Chaining Root Causes with Error Cause Properties

7.2 Symbols & Primitive Uniqueness CORE

- **7.2.1** Generating Unique Symbols with Symbol()
- **7.2.2** Global Symbol Registry: Symbol.for() & Symbol.keyFor()
- **7.2.3** Customizing Engine Hooks via Well-Known Symbols (Symbol.iterator)

7.3 Iterators, Generators & Iterable Protocols CORE

- **7.3.1** The Iterator Protocol ({ value, done })
- **7.3.2** Custom Iterables using [Symbol.iterator]
- **7.3.3** Generator Functions (function*) & yield Execution
- **7.3.4** Async Generators & for await...of Iteration

7.4 Specialized Data Collections (Map, Set, WeakMap, WeakSet) CORE

- **7.4.1** Keyed Collections: Map vs Object
- **7.4.2** Managing Unique Sets with Set
- **7.4.3** Garbage Collection Integration: WeakMap & WeakSet

7.5 Meta-Programming: Proxy & Reflect API CORE

- **7.5.1** Target Interception using Proxy Traps (get, set, apply)
- **7.5.2** Default Operation Delegation with the Reflect API
- **7.5.3** Building Reactive State Systems with Proxies

7.6 Binary Data Handling: ArrayBuffer & TypedArrays OPTIONAL

- **7.6.1** Allocating Binary Memory with ArrayBuffer
- **7.6.2** TypedArray Views (Uint8Array, Float64Array)
- **7.6.3** DataView Manipulation & Endianness Controls
- **7.6.4** String Encoding Conversions via TextEncoder & TextDecoder

---

Level 8: Engine Architecture, Browser Rendering & Performance

8.1 V8 Engine Architecture: AST, Ignition & TurboFan CORE

- **8.1.1** Source Code Parsing & Abstract Syntax Tree (AST) Generation
- **8.1.2** Ignition Interpreter Bytecode Compilation
- **8.1.3** TurboFan JIT Optimizing Compiler & Inline Caching (IC)
- **8.1.4** Identifying Engine De-optimization Anti-Patterns

8.2 Browser Rendering Pipeline & Frame Scheduling CORE

- **8.2.1** DOM ➔ CSSOM ➔ Render Tree ➔ Layout ➔ Paint ➔ Composite Pipeline
- **8.2.2** Understanding Reflow vs Repaint Triggers
- **8.2.3** Avoiding Layout Thrashing & Scheduling with requestAnimationFrame

8.3 The Event Loop In-Depth: Microtasks vs Macrotasks CORE

- **8.3.1** Call Stack & Event Loop Coordination
- **8.3.2** Microtask Queue Execution (Promises, queueMicrotask)
- **8.3.3** Macrotask Queue Execution (Timers, I/O Events)
- **8.3.4** Frame Rendering Synchronization & Task Starvation

8.4 Memory Lifecycle & Garbage Collection Mechanics CORE

- **8.4.1** Stack vs Heap Allocation Rules
- **8.4.2** Mark-and-Sweep Garbage Collection Algorithm
- **8.4.3** Generational GC: Scavenger (Young) vs Mark-Compact (Old)

8.5 Profiling Memory Leaks with DevTools Snapshots CORE

- **8.5.1** Locating Detached DOM Node Leaks
- **8.5.2** Tracing Retained Closure Scope Leaks
- **8.5.3** Heap Snapshot Analysis & Allocation Timelines

8.6 Advanced Performance & Observer APIs OPTIONAL

- **8.6.1** Lazy Loading & Viewport Tracking via IntersectionObserver
- **8.6.2** Tracking Element Bounds via ResizeObserver
- **8.6.3** Performance Metrics: Core Web Vitals (INP, LCP, CLS) & PerformanceObserver
- **8.6.4** Scheduling Idle Tasks via requestIdleCallback

🧟 Level 9: Full-Stack Integration, Build Pipelines & E2E

9.1 Server-Side JavaScript with Node.js & Express CORE

- **9.1.1** Server Execution via Node.js Runtime
- **9.1.2** Node Core Modules: fs/promises, path, http
- **9.1.3** Building REST APIs using Express / Fastify

9.2 Modern Build Tools & Bundlers (Vite, ESBuild, Babel) CORE

- **9.2.1** Asset Bundling Principles & Dev Server HMR via Vite
- **9.2.2** AST Transformations & Polyfills via Babel
- **9.2.3** Code Minification, Tree-Shaking & Source Map Generation

9.3 Advanced Test Isolation: Mocks, Spies & MSW CORE

- **9.3.1** Mocking Functions & Modules (vi.fn, vi.mock)
- **9.3.2** Spying on Methods (vi.spyOn) & Manipulating Timers
- **9.3.3** Intercepting Network Requests via Mock Service Worker (MSW)

9.4 End-to-End Automation with Playwright CORE

- **9.4.1** E2E Test Suite Setup with Playwright
- **9.4.2** Page Navigation, Input Automation & Auto-Waiting Selectors
- **9.4.3** Integrating E2E Automation into CI/CD Pipelines

🏛️ Level 10: Expert Architecture, Concurrency & Modern Standards

10.1 Web Security Architecture (XSS, CSRF, CORS, CSP) CORE

- **10.1.1** Cross-Site Scripting (XSS) Prevention & DOM Sanitization (DOMPurify)
- **10.1.2** Cross-Site Request Forgery (CSRF) Mitigation Strategies
- **10.1.3** Configuring CORS Headers & Content Security Policy (CSP) Directives

10.2 Parallel Web Concurrency (Web Workers, SharedArrayBuffer) OPTIONAL

- **10.2.1** Multi-Threaded Task Offloading via Web Workers
- **10.2.2** Message Passing & Structured Clone Algorithm
- **10.2.3** Shared Memory Access with SharedArrayBuffer & Atomics Operations

10.3 Native Web Components & Shadow DOM Encapsulation CORE

- **10.3.1** Custom Elements API & Lifecycle Callback Handlers
- **10.3.2** DOM Encapsulation via Shadow DOM (open vs closed)
- **10.3.3** Reusable UI Components using <template> & <slot> Elements

10.4 ECMAScript Standards & TC39 Proposals (ES2024–ES2026+) OPTIONAL

- **10.4.1** TC39 Governance & Stage 0 through Stage 4 Proposal Pipelines
- **10.4.2** Explicit Resource Management (using using Disposal Keyword)
- **10.4.3** Modern Temporal API for Date/Time Operations

10.5 Architectural Software Patterns in Native JavaScript CORE

- **10.5.1** Creational Design Patterns: Factory, Singleton, Builder
- **10.5.2** Structural Design Patterns: Adapter, Proxy, Decorator
- **10.5.3** Behavioral Design Patterns: Observer/PubSub, Strategy, Command
- **10.5.4** Frontend Architecture: Store Pattern & Unidirectional Data Flow